Okoro, Osahon

Learning Coach and Software Mentor

Welcome! Just to be clear from the start, this is a coaching session, not a test. It is perfectly okay, and even encouraged, to say **"I don't know"** or **"I made a mistake."** That's the fastest way we'll learn today.

I'm your supportive and non-judgmental **Learning Coach and Software Mentor**. My goal is to help you discuss your project, explain your code, identify any knowledge gaps, and strengthen your understanding. There are no grades or pass/fail outcomes in this conversation, only learning.

Before we begin, may I confirm your name? I believe it is **Osahon**. If you prefer another name, just let me know.

Since we're using a chat interface, whenever we discuss a specific piece of code, **please paste the relevant code snippet into the chat** so we can examine it together. By the end of our session, we'll have discussed all four required topics:

1. Express Routing (with a Node.js/Express snippet)
2. JavaScript Language Principles (with a JavaScript function snippet)
3. EJS Partials and Pages (with an EJS snippet)
4. Frontend CSS (with a CSS snippet)

We'll also make sure you have a transcript that you can save as a .txt file for your Canvas submission.

My first question (one question only):

Could you please confirm your name and then paste the relevant portion of your server.js file that demonstrates your Express routing so we can begin with Topic 1?

My name is Osahon Okoro. Here is the relevant portion of my server.js file that demonstrates the express routing: // == // ROUTES - Using the router from routes.js // == // Use the imported router to handle all routes app.use(router); // == // ERROR HANDLERS // == // Catch-all route for 404 errors app.use((req, res, next) => { const err = new Error('Page Not Found'); err.status = 404; next(err); }); // Global error handler app.use((err, req, res, next) => { console.error('Error occurred:', err.message); const status = err.status || 500; const context = { title: status === 404 ? 'Page Not Found' : 'Server Error', error: err.message, stack: err.stack }; res.status(status).render(errors/${status}, context); });

Thank you, **Osahon Okoro**. It's great to meet you, and thank you for sharing your code. This is a safe learning space, and remember that explaining your thought process is more important than giving a perfect answer.

Looking at your server.js snippet, I can see that you've separated your routing logic by importing a router and mounting it with app.use(router). After that, you've added middleware to catch requests that don't match any route (creating a 404 error), followed by a global error-handling middleware that renders the appropriate error page based on the status code. This organization follows a common Express pattern because it keeps the main server file clean while allowing routing logic to be maintained in separate files.

My next question (one question only):

Why did you choose to place app.use(router) before the 404 and global error-handling middleware, and what do you think would happen if the order were reversed?

i feel that is the order. because the router is like the receptionist of the company. you don't just pass with out her/him. Else this will be happening without the receptionist notification/logging. That will mean messing things up

That's a good way to think about it, Osahon. Your "receptionist" analogy captures one of the main ideas behind Express middleware.

To expand on it:

- app.use(router) gives every incoming request the opportunity to match one of your defined routes first.
- If the router finds a matching route, it sends the appropriate response (such as rendering a page), and the request is complete.
- If no route matches, Express continues to the next middleware, which in your application is the 404 handler. That middleware creates a Page Not Found error and passes it to the global error handler with next(err).
- The global error handler is intentionally last because it is responsible for handling any errors generated anywhere earlier in the middleware chain.

If the order were reversed, the 404 middleware would execute **before** the router had a chance to check whether a valid route existed. As a result, **every request, even valid ones, would immediately become a 404 error**, and your router would never get the opportunity to serve the correct page.

Your receptionist analogy works well:

- **Router:** The receptionist checks whether the visitor has a valid appointment (route).

- **404 handler:** If no appointment exists, security is notified that the visitor cannot be helped.
- **Global error handler:** Management handles the incident appropriately and provides the final response.

That explanation shows you understand not just the syntax, but also **why middleware order matters** in Express.

My next question (one question only):

For **Topic 2 (JavaScript Language Principles)**, could you please **paste a non-trivial JavaScript function from your project** that you'd like us to discuss, and then explain why you wrote it that way?

could u explain why its non trivial javascript function plz

Certainly. In this assignment, your instructor is asking for a **non-trivial JavaScript function** because they want to discuss code that demonstrates your understanding of JavaScript concepts, not just basic syntax.

A **trivial function** is very short and performs only a simple task. For example:

```
function greet(name) {
  return `Hello, ${name}`;
}
```

or

```
const add = (a, b) => a + b;
```

These functions are correct, but they don't provide much opportunity to discuss JavaScript principles.

A **non-trivial function** is one that involves some combination of:

- Decision making (if, switch)
- Loops (for, forEach, map, etc.)
- Objects or arrays
- Function parameters and return values
- Asynchronous programming (async/await, Promises)
- Error handling (try...catch)
- String or data manipulation
- Higher-order functions or callbacks

- Validation or business logic

For example, this would be considered non-trivial:

```
async function getVehicleData(req, res, next) {
  try {
    const vehicles = await invModel.getInventoryByClassificationId(
      req.params.classificationId
    );

    if (!vehicles) {
      throw new Error("No vehicles found.");
    }

    res.render("inventory/classification", {
      title: "Vehicle Inventory",
      vehicles,
    });
  } catch (error) {
    next(error);
  }
}
```

This function is non-trivial because it demonstrates several JavaScript language principles:

- async/await
- Exception handling with try...catch
- Object access (req.params)
- Variable declaration with const
- Calling another function and waiting for its result
- Conditional logic (if)
- Passing control using Express's next(error)

Another example is:

```
function buildNav(classifications) {
  let nav = "<ul>";

  classifications.forEach((item) => {
    nav += `<li><a href="/inv/type/${item.classification_id}">
```

```
            ${item.classification_name}
          </a></li>`;
  });

  nav += "</ul>";
  return nav;
}
```

This demonstrates:

- Arrays
- forEach()
- Arrow functions
- Template literals
- String concatenation
- Returning computed values

Why your instructor wants this

The goal isn't to see whether you can write JavaScript syntax. Instead, the coach wants to ask questions such as:

- Why did you choose const instead of let?
- Why did you use an arrow function?
- Why did you use async/await instead of .then()?
- Why is try...catch necessary here?
- What does this function return?
- How does Express use this function?

Those questions help demonstrate that you understand the language principles behind your code.

My next question (one question only):

Could you paste one of the larger JavaScript functions from your project (around **10–30 lines**) that you personally wrote, so we can use it for **Topic 2: JavaScript Language Principles**?

ok..Here is my non trivial code: // Global error handler app.use((err, req, res, next) => { console.error('Error occurred:', err.message); const status = err.status || 500; const context = { title: status === 404 ? 'Page Not Found' : 'Server Error', error: err.message, stack: err.stack }; res.status(status).render(errors/${status}, context); }); Express identifies error-handling

middleware by the number of parameters. If a function has 4 parameters, Express treats it as an error handler and will call it whenever next(err) is invoked anywhere in the application. If it had 3 parameters (req, res, next), Express would treat it as regular middleware and it would never receive errors.

Excellent choice, Osahon. This is definitely a non-trivial function because it demonstrates several important JavaScript and Express concepts at the same time.

What your function does

When an error is passed with next(err), Express sends the request to this middleware. Your function then:

- Logs the error message to the console.
- Determines the HTTP status code.
- Creates a context object containing data for the view.
- Renders the appropriate error page.

JavaScript principles demonstrated

Arrow Function

You used an arrow function, which provides a concise function syntax.

Template Literals

This dynamically builds the view path using JavaScript string interpolation.

Conditional (Ternary) Operator

This is a compact form of an if/else statement.

Object Creation

You create an object literal that groups related data together.

Short-Circuit Default Value

If err.status is undefined, JavaScript falls back to 500.

Your explanation is correct

You said:

Express identifies error-handling middleware by the number of parameters.

That is absolutely right. This is one of the most important Express concepts.

4 parameters:

→ Express treats it as error-handling middleware.

3 parameters:

→ Express treats it as regular middleware.

So your statement that a 3-parameter version would never receive errors passed with next(err) is correct.

One improvement you could mention in the coaching session

In production, many developers avoid sending err.stack to users because it can reveal internal server details. They might do something like:

That shows awareness of security and environment-based configuration.

My next question (one question only)

For Topic 3: EJS Partials and Pages, could you please paste an EJS snippet from your project that uses a partial (for example a header, navigation, or footer include), and then explain why you chose to use a partial instead of repeating the HTML on every page?

i used partials in my categories.ejs and here is the code: <%- include('partials/header') %> <h1>Service Project Categories</h1> <ul class="categories-list"> Environmental Educational Community Service Health and Wellness <%- include('partials/footer') %> I used partials to follow the DRY principle, so I only have to write the header and footer once, and every page automatically uses them, making the site consistent, easier to maintain, and faster to update.

That's an excellent explanation, Osahon. Your answer demonstrates that you understand both **how** EJS partials work and **why** they're useful.

Let's look at your code:

```
<%- include('partials/header') %>

<h1>Service Project Categories</h1>
<ul class="categories-list">
  <li>Environmental</li>
  <li>Educational</li>
  <li>Community Service</li>
  <li>Health and Wellness</li>
</ul>

<%- include('partials/footer') %>
```

What this code is doing

Your page is composed of three sections:

1. The **header partial** is inserted at the top.
2. The page-specific content (the categories list) is rendered.
3. The **footer partial** is inserted at the bottom.

When Express renders this EJS file, it replaces each include() statement with the contents of the corresponding partial file, creating one complete HTML page that is sent to the browser.

Your explanation is strong

You said:

"I used partials to follow the DRY principle, so I only have to write the header and footer once, and every page automatically uses them, making the site consistent, easier to maintain, and faster to update."

That is exactly what an instructor is looking for. You connected the implementation to a software engineering principle (**DRY – Don't Repeat Yourself**) instead of simply saying "because it's easier."

You could make your answer even stronger

If I were your coach during the live discussion, I'd encourage you to add one more point:

- If you later decide to add a new navigation link or change the website logo, you only need to edit **header.ejs once**. Every page that includes the header automatically reflects the change. This reduces duplication and minimizes the chance of inconsistencies or forgetting to update one page.

This demonstrates that you understand **maintainability**, which is one of the biggest advantages of using partials.

You've now covered:

- ✅ Topic 1 – Express Routing
- ✅ Topic 2 – JavaScript Language Principles
- ✅ Topic 3 – EJS Partials and Pages

You're explaining your design decisions well rather than just describing the syntax, which is exactly the goal of this coaching session.

My next question (one question only):

For **Topic 4: Frontend CSS**, could you **paste a CSS snippet from your project** that you think best demonstrates your styling approach, and explain why you chose those styles?

here is the snippet: /* ========== RESET & BASE STYLES ========== */ * { margin: 0; padding: 0; box-sizing: border-box; } body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; line-height: 1.6; color: #333; background-color: #f9f9f9; min-height: 100vh; display: flex; flex-direction: column; } /* ========== CATEGORIES LIST (GRID LAYOUT) ========== */ .categories-list { display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 1.5rem; list-style: none; padding: 0; margin: 2rem 0; } .categories-list li { background: white; padding: 1.5rem; border-radius: 8px; box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1); text-align: center; font-weight: 600; font-size: 1.1rem; color: #2c3e50; transition: transform 0.2s ease, box-shadow 0.2s ease; cursor: pointer; } .categories-list li:hover { transform: translateY(-4px); box-shadow: 0 4px 16px rgba(0, 0, 0, 0.15); } I prioritized simplicity, responsiveness, and interactivity, using a clean color scheme, CSS Grid for flexible layouts, and subtle hover animations to create a polished user experience

Excellent, Osahon. That's a well-chosen CSS snippet, and your explanation clearly shows that you were making intentional design decisions rather than just applying styles. This completes all four required topics for your coaching session.

Here's some feedback as your learning coach.

Your CSS demonstrates several important frontend principles:

- **CSS Reset:** Using the universal selector (*) to remove default margins and padding helps ensure consistent rendering across different browsers.
- **Box Model:** Setting box-sizing: border-box makes element sizing more predictable because padding and borders are included in the element's width and height.
- **Responsive Design:** The rule

grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));

is one of the strongest parts of your code. It allows the grid to automatically adjust the number of columns based on the available screen width without needing media queries.

- **User Experience (UX):** The hover effect

transform: translateY(-4px);
transition: transform 0.2s ease, box-shadow 0.2s ease;

provides visual feedback to users, making the interface feel more interactive while remaining subtle and professional.

- **Maintainability:** I also noticed your use of comment sections such as:

/* RESET & BASE STYLES */

and

/* CATEGORIES LIST (GRID LAYOUT) */

Organizing your stylesheet this way makes it much easier to navigate and maintain as the project grows.

Your explanation

You said:

"I prioritized simplicity, responsiveness, and interactivity, using a clean color scheme, CSS Grid for flexible layouts, and subtle hover animations to create a polished user experience."

That's an excellent answer because it explains **the design goals** behind your CSS rather than simply listing the properties you used. During an oral discussion, that's exactly what an instructor wants to hear.

Coaching Session Summary

Over the course of this session, you demonstrated understanding of all four required topics:

- ✅ **Express Routing:** You explained why app.use(router) must come before the 404 and global error handlers and correctly described the importance of middleware order.
- ✅ **JavaScript Language Principles:** You discussed a non-trivial error-handling middleware and correctly explained why Express identifies it by its four parameters.
- ✅ **EJS Partials and Pages:** You showed how using partials supports the DRY principle and improves consistency and maintainability.
- ✅ **Frontend CSS:** You explained your use of CSS Grid, responsive layouts, hover effects, and organized styling to create a clean user experience.

Throughout the discussion, you consistently explained **why** you made your design choices, which is exactly the purpose of this coaching activity.

Since we've naturally completed all four required discussion topics, this concludes our coaching session. There is no pass or fail, our goal was simply to deepen your understanding, and you've done a good job articulating the reasoning behind your implementation choices.

For your Canvas submission, simply save this conversation as a .txt or .md file (or use ChatGPT's export/download feature if available) and upload it as your coaching session transcript.

Well done, Osahon, and I wish you the best with your CSE 340 course!